Manual de ejercicios para la materia Lenguajes de Programación

Luis Miguel Muñoz Barón

Facultad de Ciencias UNAM

Licenciatura en Ciencias de la Computación

Tutor: Javier Enriquez Mendoza

7 de Abril 2026

Contenido

1. Motivación & Objetivos
2. Propuesta didáctica
3. Contenido

Motivación

1. **¿Qué dificultades tienen los alumnos?**
2. **¿Qué huecos hay en los materiales existentes?**

limitaciones lectivas y desafíos comunes

1. Dificultad para aplicar reglas formales en ejecuciones abstractas
2. Problemas para entender semántica operacional vs denotacional[1]
3. Confusión de conceptos como variables libres, alcance de variable, variable ligada[2]
4. Confusión en λ-cálculo notación, sustitución y α-equivalencia[3]

1. Resources for practice with compositional semantics? r/asklinguistics
2. How does it help to read "An introduction to Functional Programming Through Lambda Calculus"?
3. Lambda Calculus made easy r/math

limitaciones lectivas y desafíos comunes

1. Alta aplicación de razonamiento lógico estructurado[4]
2. Manipulación de notación sin entendimiento sobre su significado
3. Dificultad de aprendizaje autónomo y progresivo[5]
4. Incapacidad de transferir teoría a código real[6]

4. Students Struggle to Explain Their Own Program Code
5. RFC: Tackling learners' struggles in Core Programming Issue 1094
6. Improving Concept Learning Through Specialized Digital Fanzines

Objetivo General

Desarrollar un material didáctico autocontenido que facilite la comprensión y práctica de conceptos fundamentales en lenguajes de programación.

Objetivos específicos

- Presentar teoría clara y estructurada
- Diseñar ejercicios de complejidad progresiva
- Fomentar el aprendizaje autónomo
- Reforzar la comprensión conceptual

Propuesta didáctica

Sección teórica:

- Objetivo(s)
- Planteamiento
- Definiciones
- Intuición y formalización
- Ejemplos resueltos

Sección teórica:

- Ejemplos abiertos para el lector

Estructura del material

- Introducción, Gramáticas y Herramientas Matemáticas
- Sintaxis y Semántica
- Cálculo Lambda y MiniHs
- Inferencia de tipos
- TinyC & Máquinas Abstractas
- Featherweight Java

Contenido

Algunos Ejemplos

Ejercicio (4.5 Capítulo 4)

La sintáxis para definir una calculadora utilizando la notación polaca reversa (NPR) se define como:

$$\frac{x \in \mathbb{N}}{x\ Symbol} \quad \frac{x \in \{+, -, *, /\}}{x\ Symbol} \quad \frac{}{\epsilon \quad \texttt{NPR}} \quad \frac{x\ Symbol\ xs \quad \texttt{NPR}}{x\ xs \quad \texttt{NPR}}$$

$7\ +\ 1$ *se representa como:* $7\ 1\ +$

$7\ -\ (3\ +\ 2)$ *se representa como:* $7\ 3\ 2\ +\ -$

Ejercicio (4.5 Capítulo 4)

Esta gramática tiene el problema de poder formar expresiones que no pertenecen necesariamente a ***NPR*** *como:*

$$1 + 2$$

$$+ * /$$

Ejercicio (4.5 Capítulo 4)

- *Proporciona un conjunto de reglas para definir la semántica estática que pueda analizar una expresión e y nos diga si una expresión perteneciente a* `NPR` *está bien formada, denotando el juicio como:* $\vdash e\ Ok$.

Ejercicio (4.5 Capítulo 4)

Solución:

El programa vacío es válido solo si hay exactamente un valor en la pila; esto significa que el programa evaluará a un único resultado:

$$1 \vdash \epsilon \; Ok \quad \textsc{Vacío}$$

Semántica estática para NPR

Ejercicio (4.5 Capítulo 4)

Anteponer un número a un programa implica que se requiere un valor menos en la pila:

$$\frac{x \in \mathbb{N} \quad (n+1) \vdash xs\ Ok}{n \vdash x\ xs\ Ok}\ \textsc{Num}$$

Anteponer un operador requiere dos valores en la pila y produce uno. La estructura de la regla refleja esto:

$$\frac{x \in \{+, -, *, /\} \quad (n+1) \vdash xs\ Ok}{(n+2) \vdash x\ xs\ Ok}\ \textsc{Op}$$

Derivación estática: 3 2 1 + −

Ejercicio

Aplica el analizador sintáctico para la expresión:

$$3\ 2\ 1\ +\ -$$

$$\dfrac{3 \in \mathbb{N} \qquad \dfrac{2 \in \mathbb{N} \qquad \dfrac{1 \in \mathbb{N} \qquad \dfrac{+ \in \{+,-,*,/\} \qquad \dfrac{- \in \{+,-,*,/\} \qquad 1 \vdash \epsilon\ Ok\ \textsc{Vacío}}{2 \vdash -\ \epsilon\ Ok}\ \textsc{Op}}{3 \vdash +\ -\ \epsilon\ Ok}\ \textsc{Op}}{2 \vdash 1\ +\ -\ \epsilon\ Ok}\ \textsc{Num}}{1 \vdash 2\ 1\ +\ -\ \epsilon\ Ok}\ \textsc{Num}}{0 \vdash 3\ 2\ 1\ +\ -\ \epsilon\ Ok}\ \textsc{Num}$$

Semántica dinámica para NPR

Ejercicio (4.5 Capítulo 4)

- *Proporciona una conjunto de reglas para implementar la semántica operacional de paso pequeño para las expresiones bien formadas de* `NPR`*. Define la relación* $\rightarrow : Q \times Q$*, que evalúa un paso del cálculo.*

Ejercicio (4.5 Capítulo 4)

Solución:

Nuestros estados Q serán de la forma $s \triangleright p$, donde s es una pila de números naturales y p es un programa en notación polaca reversa (NPR).

Los estados iniciales son todos aquellos de la forma $\bullet \triangleright p$.

Los estados finales son todos aquellos de la forma $n \triangleright \epsilon$.

Semántica dinámica para NPR

Ejercicio (4.5 Capítulo 4)

- *Proporciona una conjunto de reglas para implementar la semántica operacional de paso pequeño para las expresiones bien formadas de* `NPR`*. Define la relación* $\rightarrow : Q \times Q$*, que evalúa un paso del cálculo.*

Semántica de paso pequeño para RPN

Ejercicio (4.5 Capítulo 4)

Los números simplemente se apilan en la pila:

$$\frac{x \in \mathbb{N}}{s \ \triangleright \ x \ s' \ \rightarrow \ x : s \ \triangleright \ s'} \quad \textit{SS-Num}$$

Las operaciones extraen dos elementos de la pila y operan sobre ellos:

$$\frac{x \in \{+, -, *, /\}}{a : b : s \ \triangleright \ x \ s' \ \rightarrow \ (a \ x \ b) : s \ \triangleright \ s'} \quad \textit{SS-Op}$$

Semántica de paso pequeño para RPN

Ejercicio (4.5 Capítulo 4)

Evalúa la siguiente expresión: $\bullet \triangleright 1\ 5\ 10\ /\ -$

$$
\begin{array}{lll}
\rightarrow & 1 : \bullet \triangleright 5\ 10\ /\ - & \textit{SS-Num} \\
\rightarrow & 5 : 1 : \bullet \triangleright 10\ /\ - & \textit{SS-Num} \\
\rightarrow & 10 : 5 : 1 : \bullet \triangleright\ /\ - & \textit{SS-Num} \\
\rightarrow & (10/5) : 1 : \bullet \triangleright\ - & \textit{SS-Op} \\
\rightarrow & 2 : 1 : \bullet \triangleright\ - & \\
\rightarrow & (1-2) : \bullet \triangleright \epsilon & \textit{SS-Op} \\
\rightarrow & -1 \triangleright \epsilon &
\end{array}
$$

Temas estudiados en este ejercicio

- Sintáxis concreta.
- Gramáticas permisivas.
- Análisis estático y reglas de inferencia.
- Semántica operacional de paso pequeño.
- relaciones de transición.
- Evaluación de expresiones.

Ejercicio

Supón ahora que se requiere implementar una máquina abstracta para modelar la ejecución de las expresiones bien formadas a partir de la sintáxis concreta del Cálculo Lambda junto con el operador "+".

Ejercicio

- *Propón un conjunto de reglas para implementar un proceso de análisis estático para expresiones cerradas.*
- *Propón un conjunto de reglas para implementar la semántica dinámica para evaluar expresiones.*

Ejercicio

Solución:

$$\frac{n \in Int}{\Gamma \vdash (\mathit{Num}\ n)\ ok} \qquad \frac{x \in \Gamma}{\Gamma \vdash x\ ok}$$

$$\frac{\Gamma, x \vdash e\ ok}{\Gamma \vdash (\mathit{Lambda}\ x.e)\ ok} \qquad \frac{\Gamma \vdash e_1\ ok \quad \Gamma \vdash e_2\ ok}{\Gamma \vdash (\mathit{Apply}\ e_1\ e_2)\ ok}$$

$$\frac{\Gamma \vdash e_1\ ok \quad \Gamma \vdash e_2\ ok}{\Gamma \vdash (\mathit{Plus}\ e_1\ e_2)\ ok}$$

Derivación: Expresión Cerrada

Ejercicio

Sea:

$$e = (\mathit{Apply}\ (\mathit{Lambda}\ x.\ \mathit{Plus}\ x\ \mathit{Num}\ 1)\ 2)$$

$$\dfrac{\dfrac{\dfrac{\dfrac{x \in \{x\}}{\{x\} \vdash x\ \ ok} \qquad \dfrac{}{\{x\} \vdash \mathit{Num}\ 1\ \ ok}}{\{x\} \vdash \mathit{Plus}\ x\ \ \mathit{Num}\ 1\ \ ok}}{\emptyset \vdash \mathit{Lambda}\ x.\mathit{Plus}\ x\ \ \mathit{Num}\ 1\ \ ok} \qquad \dfrac{}{\emptyset \vdash \mathit{Num}\ 2\ \ ok}}{\emptyset \vdash (\mathit{Apply}\ (\mathit{Lambda}\ x.\ \mathit{Plus}\ x\ \mathit{Num}\ 1)\ \mathit{Num}\ 2)\ ok}$$

Semántica Operacional de paso pequeño

Ejercicio

$$\frac{e_1 \mapsto e_1'}{(\mathit{Apply}\ e_1\ e_2) \mapsto (\mathit{Apply}\ e_1'\ e_2)}$$

$$\frac{}{(\mathit{Apply}\ (\mathit{Lambda}\ x.e_1)\ e_2) \mapsto e_1[x := e_2]}$$

$$\frac{e_1 \mapsto e_1'}{(\mathit{Plus}\ e_1\ e_2) \mapsto (\mathit{Plus}\ e_1'\ e_2)}$$

$$\frac{e \mapsto e'}{(\mathit{Plus}\ (\mathit{Num}\ n)\ e) \mapsto (\mathit{Plus}\ (\mathit{Num}\ n)\ e')}$$

$$\frac{}{(\mathit{Plus}\ (\mathit{Num}\ n_1)\ (\mathit{Num}\ n_2)) \mapsto (\mathit{Num}\ (n_1 + n_2))}$$

Inciso (a)

Ejercicio

Iniciales:

$$S = \{e \mid \emptyset \vdash e \ ok\}$$

Finales:

$$F = \{\mathsf{Num}\ n,\ \mathsf{Lambda}\ x.e\}$$

Inciso (b): Ejecución

Ejercicio

Evaluar:

$$(\mathit{Apply}\ (\mathit{Lambda}\ x.\ \mathit{Plus}\ x\ (\mathit{Num}\ 1))\ (\mathit{Num}\ 2))$$

Ejecución paso a paso

Ejercicio

$$(\mathit{Apply}\ (\mathit{Lambda}\ x.\ \mathit{Plus}\ x\ \mathit{Num}\ 1)\ \mathit{Num}\ 2) \mapsto$$

$$(\mathit{Plus}\ x\ \mathit{Num}\ 1)[x := \mathit{Num}\ 2] \mapsto$$

$$(\mathit{Plus}\ (\mathit{Num}\ 2)\ (\mathit{Num}\ 1)) \mapsto$$

$$(\mathit{Num}\ (2 + 1)) \mapsto$$

$$(\mathit{Num}\ 3)$$

Ejercicio (9.11 Capítulo 5)

Supón que se requieren extender las estructuras de datos para el Cálculo Lambda con la introducción de árboles binarios.

- *Define el árbol binario vacío $\varnothing$ (Empty).*
- *Define el constructor de un nodo que contiene un elemento, un sub-árbol izquierdo y derecho. (Branch x l r)*
- *Define la función* fold *que nos permita iterar el árbol y computar alguna operación sobre sus elementos.*

Definición del árbol vacío

Ejercicio (9.11 Capítulo 5)

El árbol vacío se define como:

$$\mathsf{Empty} = \lambda e \lambda b.\ e$$

- *e representa el caso base (árbol vacío).*
- *b representa el caso de nodo.*

Constructor de nodos

Ejercicio (9.11 Capítulo 5)

Un nodo con valor x, subárbol izquierdo l y derecho r:

$$\mathsf{Branch}\ x\ l\ r = \lambda e \lambda b.\ b\ x\ (l\ e\ b)\ (r\ e\ b)$$

- *x es el valor almacenado.*
- *l y r son subárboles.*

Ejercicio (9.11 Capítulo 5)

Consideremos el árbol:

$$\mathsf{Branch}\ 1\ \mathsf{Empty}\ (\mathsf{Branch}\ 2\ \mathsf{Empty}\ \mathsf{Empty})$$

Aplicando las definiciones:

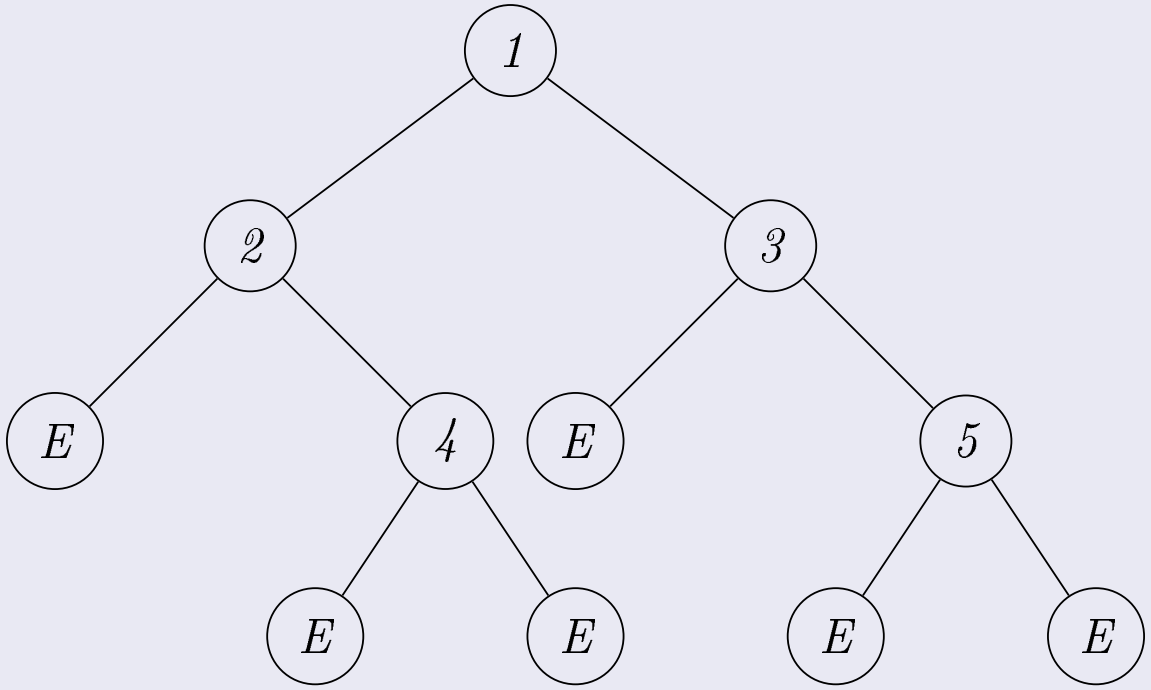
- $\mathsf{Empty} = \lambda e \lambda b.\ e$
- $\mathsf{Branch}\ x\ l\ r\ = \lambda e \lambda b.\ b\ x\ (l\ e\ b)\ (r\ e\ b)$

Se reduce a:

$$\lambda e \lambda b.\ b\ 1\ e\ (b\ 2\ e\ e)$$

Árbol asociado a `Branch`

Ejercicio (Ejemplo de 4 niveles)



$$\textit{Branch}\ 1\ (\textit{Branch}\ 2\ \textit{Empty}\ (\textit{Branch}\ 4\ \textit{Empty}\ \textit{Empty}))$$
$$(\textit{Branch}\ 3\ \textit{Empty}\ (\textit{Branch}\ 5\ \textit{Empty}\ \textit{Empty}))$$

$$\lambda e \lambda b.\ b\ 1\ (b\ 2\ e\ (b\ 4\ e\ e))\ (b\ 3\ e\ (b\ 5\ e\ e))$$

Definición de fold

Ejercicio (9.11 Capítulo 5)

La función **fold** *sobre árboles se define como:*

$$\mathsf{fold}\ t\ e\ op = t\ e\ op$$

Ejemplo de uso de fold

Ejercicio (9.11 Capítulo 5)

Ejemplo: sumar todos los elementos del árbol.

$$\mathsf{fold}\ t\ 0\ (\lambda x \lambda l \lambda r.\ x + l + r)$$

Evaluación con fold (suma)

Ejercicio (9.11 Capítulo 5)

$$t =_{def} \lambda e \lambda b.\ b\ 1\ e\ (b\ 2\ e\ e)$$

Aplicamos fold *con suma:*

$$t\ 0\ (\lambda x\ l\ r.\ x + l + r)$$

$$\begin{aligned}
& (\lambda e\ b.\ b\ 1\ e\ (b\ 2\ e\ e))\ 0\ (\lambda x\ l\ r.\ x + l + r) \\
\to_\beta\ & (\lambda b.\ b\ 1\ 0\ (b\ 2\ 0\ 0))\ (\lambda x\ l\ r.\ x + l + r) \\
\to_\beta\ & (\lambda x\ l\ r.\ x + l + r)\ 1\ 0\ ((\lambda x\ l\ r.\ x + l + r)\ 2\ 0\ 0) \\
\to_\beta\ & (\lambda x\ l\ r.\ x + l + r)\ 1\ 0\ (2 + 0 + 0) \\
\to_\beta\ & 1 + 0 + (2 + 0 + 0) \\
\to_\beta\ & 3
\end{aligned}$$

Conclusiones

Limataciones y alcance del material

- hola

Gracias!